

GOVERNMENT OF DUBAI MEDIA OFFICE

JotFlow

[workflow_assign_form Access Link](#)
Implementation Plan

Prepared for: Government of Dubai Media Office (GDMO)

Date: June 14, 2026

Version: 1.0

Classification: Confidential

1. Executive Summary

GDMO is migrating from JotForm Inbox to JotFlow as the primary workflow interface. The core promise: users will see all their assigned tasks, forms, and approvals directly in JotFlow — no need to check email.

JotForm workflows contain 3 types of steps:

- **workflow_approval** (approve/reject with signature) — Already fully handled in-app
- **workflow_assign_task** (complete a task) — Already fully handled in-app
- **workflow_assign_form** (fill a separate form) — **THE REMAINING CHALLENGE**

This document covers the implementation strategy for `workflow_assign_form` — the only task type that requires an external link to JotForm.

2. The Problem

When a workflow assigns a form to someone, JotForm creates a **prefilled URL** that:

- Opens the correct target form
- Pre-populates fields from the parent submission (e.g., LPO, Invoice, Project Description)
- Authenticates the assignee
- Marks the workflow task as completed when the form is submitted

Without the correct URL (specifically the `prefillID`), the form opens without pre-populated data, fields are locked, and the form cannot be submitted.

The JotForm API does **NOT** expose this prefill URL directly. Previously, the only way to get it was harvesting it from JotForm's email system — which has a **6-day retention window**.

3. The Solution — Prefills API

3.1 Discovery

Through API testing against the live GDMO instance, we discovered that the JotForm Prefills API (GET /form/{formID}/prefills) returns ALL prefill URLs ever created for a form, including those generated by workflow task assignments.

3.2 URL Pattern (Confirmed Across Multiple Forms, Assignees, and Dates)

```
https://eforms.mediaoffice.ae/{formID}/prefill/{prefillID}?
workflowAssignFormTask=1&taskID={taskID}
```

This pattern is consistent across:

- 4 different emails tested
- 2 different forms (POC form 260825008947058, GDMO form 250371191810045)
- Multiple assignees (internal GDMO staff + external)
- Dates spanning March–June 2026

No extra filters, conditions, or parameters are appended. The pattern is fixed.

3.3 How to Construct the URL

All 3 components are available from JotForm APIs:

Component	Source	API Endpoint
formID	element.formID in task data	GET /workflow/submission/{id}/tasks
taskID	task.id in task data	GET /workflow/submission/{id}/tasks
prefillID	urls[].id where settings.id = parent submissionID	GET /form/{formID}/prefills

Matching Logic:

1. Call GET /form/{targetFormID}/prefills — returns all prefill URL objects
2. Filter: find URL entries where urls[].settings.id === parentSubmissionID

3. If multiple matches exist (JotForm creates new ones for reminders), pick the latest by `created_at`
4. The `url.id` field is the `prefillID`

3.4 Why This Works

- The Prefills API retains ALL URLs (no 6-day expiry like emails)
- One API call per form returns data for ALL submissions (587 URLs returned for the real GDMO form in testing)
- The `settings.id` field reliably maps to the parent submission ID
- The `taskID` is available from the workflow tasks API

4. Implementation Architecture

4.1 For New Submissions (Going Forward)

JotForm assigns form task

|

v

Webhook POST /api/webhook fires

|

v

Backend fetches GET /workflow/submission/{id}/tasks

|

v

For workflow_assign_form tasks:

-> Extract formID and taskID from task data

-> Call GET /form/{formID}/prefills

-> Match prefillID by settings.id = submissionID

-> Construct URL: {host}/{formID}/prefill/{prefillID}?workflowAssignFormTask=1&taskID={taskID}

-> Store in workflow_tasks JSONB as accessLink

|

v

Frontend shows "Open Form" button with the constructed URL

4.2 For Existing Submissions (14k Migration Backfill)

Backfill Script (one-time):

```

|
v
1. Query all jf_submissions with non-completed workflow_assign_form tasks
|
v
2. Group by target formID (to minimize API calls)
|
v
3. For each unique target formID:
  -> Call GET /form/{formID}/prefills (one call returns ALL URLs)
  -> Build lookup map: parentSubmissionID -> latest prefillID
  |
  v
4. For each submission:
  -> Look up prefillID from the map
  -> Get taskID from workflow_tasks JSONB or GET /workflow/submission/{id}/tasks
  -> Construct URL
  -> UPDATE jf_submissions SET workflow_tasks = (updated JSONB with accessLink)
  |
  v
5. Log results: matched vs unmatched submissions

```

Efficiency: For 14k submissions, the backfill requires approximately N API calls to GET /form/{formID}/prefills (where N = number of unique target forms, likely 10–50), plus M calls to GET /workflow/submission/{id}/tasks (only for submissions missing taskID). Total: likely under 200 API calls for the entire migration.

4.3 Background Poller Integration

The existing poller (runs every ~2 minutes) will be enhanced:

1. After fetching workflow tasks for a submission, check if any are workflow_assign_form with empty accessLink
2. If found, fetch prefills for that formID (cached per form, 10-minute TTL)
3. Match and store the prefill URL
4. This ensures any new form assignments get their URL within ~2 minutes

5. Fallback Mechanism (Defense in Depth)

If the primary Prefills API approach fails, there are 4 fallback layers:

Layer 1: Prefills API PRIMARY

- **Reliability:** HIGH — API retains all URLs, no expiry
- **Failure mode:** API down or form deleted

Layer 2: Email Harvesting EXISTING — ENHANCED

- **How:** Scan enterprise/system-logs for form assignment emails, extract prefill URL from email body
- **Reliability:** MEDIUM — 6-day email retention window
- **When used:** If Prefills API returns no match for a submission
- **Enhancement:** Update the regex pattern to also match `/prefill/` URLs (currently only matches `/share/` URLs, which is wrong for `assign_form` tasks)

Layer 3: Constructed URL without prefillID LOW RELIABILITY

- **Pattern:** `{host}/{formID}?workflowAssignFormTask=1&taskID={taskID}`
- **Reliability:** LOW — form opens but fields are not pre-populated, may not be submittable
- **When used:** If both Prefills API and email harvesting fail
- **User experience:** Shows a warning that the form may not have pre-filled data

Layer 4: Inbox Fallback GUARANTEED

- **Pattern:** `{host}/inbox/{sourceFormID}/{submissionID}`
- **Reliability:** GUARANTEED — always works
- **When used:** Last resort only
- **User experience:** Shows submission details in read-only view; user may need to contact admin

Fallback Decision Flow

Has prefillID from Prefills API?

-> YES -> Construct full prefill URL

[OK]

-> NO -> Is email within 6-day window?

-> YES -> Harvest from email body

[OK]

-> NO -> Do we have formID + taskID?

-> YES -> Construct URL without prefillID

[WARNING]

-> NO -> Inbox fallback (with admin alert)

[MANUAL]

6. Migration Plan

Phase 1: Pre-Migration Preparation (1–2 days)

- Deploy updated backend with Prefills API integration
- Run backfill script on staging database
- Verify URL construction for sample submissions across different forms
- Fix email harvester regex to also catch `/prefill/` URLs

Phase 2: Data Backfill (1 day)

- Run backfill script against production database
- Expected: 14k submissions processed, ~N form-level API calls
- Log all unmatched submissions for manual review
- Verify sample URLs work by opening them in browser

Phase 3: Cutover

- Enable enhanced poller (auto-fetches prefill URLs for new assignments)
- Update frontend to show "Open Form" button for `workflow_assign_form` tasks
- Monitor email harvester as supplementary backup
- Dashboard shows link availability stats for admin visibility

Phase 4: Post-Migration Monitoring (1 week)

- Track: % of `workflow_assign_form` tasks with valid `accessLink`
- Alert: if any new task goes >5 minutes without an `accessLink`
- Audit: compare constructed URLs vs email URLs for accuracy

7. Reliability Assessment

What Can Go Wrong and How We Handle It

Scenario	Impact	Mitigation
Prefills API is temporarily down	New tasks don't get URL immediately	Poller retries every 2 min; email harvester as backup
Form is deleted from JotForm	Prefills API returns empty	Inbox fallback; admin notified
Submission has no prefill match	URL can't be constructed	Email harvesting; constructed URL without prefillID (with warning)
Multiple prefill URLs for one submission	Could pick wrong one	Always use latest by <code>created_at</code> ; all prefills for same submission work the same way
JotForm changes URL pattern	Constructed URLs break	Monitor: compare constructed vs email-harvested URLs weekly
API rate limiting (429)	Backfill slows down	Throttled requests with <code>retry-after</code> ; batch by form to minimize calls

Expected Reliability

- **New submissions (going forward):** >99% — Prefills API is reliable, poller runs every 2 min
- **Migration backfill (14k):** >95% — depends on how many have prefill entries; remainder handled by email harvesting or manual review
- **Edge cases (<1%):** Forms deleted, workflows reconfigured, very old submissions — inbox fallback

8. API Calls Summary

Per-Submission (Real-Time)

Call	When	Cost
GET /workflow/submission/{id}/tasks	Every poll cycle for pending submissions	1 call per submission
GET /form/{formID}/prefills	Once per form, cached 10 min	1 call per unique form

Migration Backfill (One-Time)

Call	Count	Notes
GET /form/{formID}/prefills	~10–50 (one per unique target form)	Each returns ALL prefill URLs
GET /workflow/submission/{id}/tasks	Only for submissions missing taskID	Subset of 14k

Total estimated API calls for migration: <200

9. Key Technical Details

Prefill URL Object Structure

From GET /form/{formID}/prefills :

```
{
  "id": "6a2d0a3c6636339464804f0fd288",           // <-- This is the prefillID for the URL
  "prefill_id": "260824768641061",
  "form_id": "260825008947058",
  "created_at": "2026-06-13 03:43:56",
  "settings": {
    "id": "6571458336016798785",                 // <-- This is the parent submissionID
    "metadata": {
      "email": "assignee@example.com",
      "name": "assignee@example.com"
    },
    "settings": {
      "fieldBehaviour": "readonly"
    }
  }
}
```

Workflow Task Object Structure

From GET /workflow/submission/{id}/tasks :

```
{
  "id": "18271261631343598463",                 // <-- This is the taskID
  "status": "ACTIVE",
  "element": {
    "type": "workflow_assign_form",
    "formID": 260825008947058,                   // <-- This is the target formID
    "prefillEnabled": "Yes",
    "prefillFormID": 260824432509052             // <-- This is the source form
  },
  "properties": {
    "assigneeEmail": "user@example.com",
    "submissionID": "6571458926019158170"       // <-- Child submission (not needed for
URL)
  }
}
```

Constructed URL

```
https://eforms.mediaoffice.ae/260825008947058/prefill/6a2d0a3c6636339464804f0fd288?  
workflowAssignFormTask=1&taskID=18271261631343598463
```

10. Files to Modify

File	Change
backend/lib/poller.js	Add prefill URL resolution after workflow task enrichment
backend/lib/email-link-harvester.js	Update regex to also match /prefill/ URLs
backend/lib/workflow-task.js	Extract formID from element for assign_form tasks
backend/routes/forms-workflow.js	Update /api/email-url to use Prefills API as primary source
backend/routes/submissions-actions.js	No change needed (approval/task completion already works)
backend/scripts/backfill-prefill-urls.js	New script for one-time migration backfill
frontend/src/config/currentUser.ts	No change needed (getMyActionType already handles assign_form)
frontend/src/components/SubmissionModal.tsx	Show "Open Form" button with prefill URL for assign_form tasks